Week 9 슬라이드 자습 가이드

Memory Pressure and Page Reclaim: Watermarks, kswapd, LRU, Swap, PSI, Memory Cgroup · Sungyong Park, Ph.D. (DISCOS Lab, Sogang University)

이 자료를 사용하는 방법

이 자료는 Week 9 슬라이드(총 50쪽)를 보면서 함께 읽는 자습 가이드입니다. 슬라이드는 영어로, 설명은 한국어로 작성했습니다. 슬라이드마다 내용 요약과 기억할 점을 정리했습니다. 1, 4, 17, 34, 50쪽은 표지, 구간 안내, 마지막 슬라이드라서 표에 넣지 않았습니다.

이번 주 학습 목표. 수업이 끝나면 다음 세 가지를 말로 설명할 수 있어야 합니다.

1. memory pressure가 무엇인지, watermark(min/low/high)가 kswapd와 direct reclaim을 언제 시작시키는지, direct reclaim이 지연을 늘리는 이유
2. reclaim이 어떤 페이지를 먼저 고르고(LRU, 비용과 재사용), 한 번의 reclaim round가 어떻게 진행되며, swap, zram, shrinker가 어떤 역할을 하는지
3. PSI, /proc/vmstat, memory cgroup으로 압박의 심각도, 경로, 소유자를 관찰하는 방법과 튜닝의 대가, 작은 메모리 기기(webOS급)에서의 설계 방향

수업 구성. 세 가지 질문으로 진행합니다. 1부는 Linux가 언제, 누가 메모리를 reclaim하는지(5–16쪽), 2부는 어떤 페이지를 먼저 reclaim하고 어떻게 해제하는지(18–33쪽), 3부는 메모리 압박을 어떻게 관찰하고 제어하고 진단하는지(35–48쪽)를 다룹니다. 49쪽은 한 장 요약입니다.

추천 학습 방법.

1. 슬라이드 왼쪽의 Key Points를 읽고, 아래의 Key Takeaway를 다시 한 번 읽습니다.
2. 이 자료의 요약과 "기억할 점"으로 이해한 내용을 확인합니다.
3. 부가 끝날 때마다 Checkpoint 슬라이드(16, 33, 48쪽)의 질문에 먼저 스스로 답해 보고, 마지막 장의 답과 비교합니다.
4. 슬라이드에 "Week 10"이라고 표시된 내용(OOM killer, Low Memory Killer)은 10주차에서 자세히 다룹니다. 이번 주에는 이해하지 못해도 괜찮습니다.

참고 사항. 예시 출력과 수치는 Linux 6.x 기준이며, 관찰 슬라이드(39–41쪽)의 값은 2 CPU, 8 GB RAM 테스트 VM에서 얻은 한 번의 실행 결과입니다. 그 밖의 수치(16 GB 메모리 구성, 지연 시간, cgroup 관찰값 등)는 설명용 예시입니다. 개수와 시간 같은 수치는 커널 버전, 설정, 환경에 따라 달라집니다.

쪽	슬라이드	내용 요약	기억할 점
2	**Roadmap of Part 3**	Part 3은 4주 구성입니다. 7주차는 애플리케이션 관점의 가상 메모리, 8주차는 커널 관점의 주소 변환과 물리 메모리 할당(page table, TLB, page fault, buddy, slab)입니다. 9주차(이번 주)는 memory pressure와 page reclaim, 10주차는 reclaim이 따라가지 못할 때의 OOM killer, LMK와 원인 분석을 다룹니다.	이번 주는 "메모리가 부족해지면 Linux가 어떻게 자유 메모리를 다시 만드는가"입니다. 부족을 넘어선 뒤의 일은 10주차입니다.
3	**Where Does Week 9 Fit in the Page Allocation Path?**	페이지 할당은 세 단계입니다. fast path(8주차: free page가 있으면 바로 할당), slow path(이번 주: 할당이 어려우면 사용 중인 메모리를 reclaim해 free memory를 확보한 뒤 할당), failure(10주차: OOM killer, Low Memory Killer)입니다. reclaim은 일과 대기가 필요하므로 slow path가 fast path보다 느립니다. 이번 주는 누가 언제 reclaim하는지, 무엇을 먼저 reclaim하는지, 어떻게 측정하고 제어하고 진단하는지로 이어집니다.	실제 경로는 GFP flag, 할당 order, reclaim 상태에 따라 달라지는 단순화된 그림입니다.

1부. Linux는 언제, 누가 메모리를 reclaim할까요? (5–16쪽)

이 블록의 핵심은 다음과 같습니다. 메모리 부족은 free의 양이 아니라 할당 비용으로 정의합니다. watermark가 reclaim을 일찍 시작시키고, kswapd가 백그라운드에서 따라잡지 못하면 할당 task가 직접 reclaim하며 기다립니다.

쪽	슬라이드	내용 요약	기억할 점
5	**How Do Allocation and Reclaim Form a Feedback Loop?**	할당은 buddy allocator의 free page를 가져가고, reclaim은 page cache, slab, anonymous 페이지를 해제해 free list에 되돌립니다. free page가 줄면 reclaim이 시작되고, reclaim이 채우면 다시 할당이 가져가는 되먹임(feedback) 고리입니다.	reclaim이 계속 채우므로 건강한 시스템에서도 free memory는 낮게 유지됩니다. free만 보고는 부족 여부를 알 수 없습니다.
6	**Why Can Low Free Memory Be Normal?**	Linux는 남는 RAM을 page cache로 씁니다. 예시(16 GB): free 1 GB, page cache 8 GB, 애플리케이션 4 GB, 커널 3 GB입니다. 애플리케이션이 2 GB를 더 할당하면 free는 그대로이고 page cache가 2 GB 줄어듭니다(reclaim). 캐시는 디스크 읽기를 줄이고, 필요하면 Linux가 자동으로 회수합니다.	"used"는 "쓸 수 없음"이 아닙니다. 수치는 설명용 예시입니다.
7	**What Is Memory Pressure in Linux?**	memory pressure는 새 할당이 더 많은 reclaim 일이나 대기를 요구하는 정도입니다. 낮음(free나 reclaimable page가 충분, 할당이 빠름), 중간(free가 줄어 kswapd가 백그라운드 reclaim, 대부분 성공), 높음(백그라운드로 부족해 요청한 task가 직접 reclaim, 지연이 길어짐)으로 점진적으로 커집니다. 지연은 마이크로초에서 밀리초, 초 단위까지 늘어납니다(예시).	부족은 free의 양이 아니라 할당의 "비용"으로 정의합니다.
8	**Which Zone Serves an Allocation Request?**	노드의 메모리는 주소에 따라 DMA, DMA32, Normal zone으로 나뉩니다. 요청은 쓸 수 있는 가장 높은 zone을 지정하고 (GFP_KERNEL은 Normal까지, GFP_DMA32는 DMA32까지, GFP_DMA는 DMA만), 그 zone부터 낮은 zone으로만 내려가며 시도합니다. low watermark 위의 zone이 있으면 fast path이고, 없으면 slow path입니다. free page와 watermark는 zone마다 따로 관리합니다.	fallback은 아래쪽으로만 갑니다. zone 구성과 크기는 아키텍처마다 다르고 노드마다 따로 있습니다.
9	**How Does Linux Reclaim Free Memory?**	reclaim은 zone별 watermark를 기준으로 합니다. kswapd(노드마다 하나인 커널 스레드)는 쓸 수 있는 zone 중 low 위가 없을 때 깨어나 high에 도달하면 멈추며, 할당 task는 대개 기다리지 않습니다. 할당이 몰리면 kswapd를 앞지를 수 있습니다. 쓸 수 있는 모든 zone이 min 아래이고 대기할 수 있는 요청이면 task가 직접 reclaim을 하고 기다립니다. atomic 요청은 기다릴 수 없어 min 아래 reserve를 씁니다.	kswapd는 백그라운드, direct reclaim은 할당 task 안에서 일어나며 task가 기다리므로 지연이 늘어납니다.
10	**Why Does Direct Reclaim Hurt Latency?**	direct reclaim은 요청한 task 안에서 돌고, 끝나야 task가 계속됩니다. 스캔(CPU), dirty page의 write-back 대기, swap-out이 더해져 할당 지연이 정상 할당(보통 마이크로초)에서 밀리초~수백 밀리초로 늘고, tail latency(p99)가 특히 나빠집니다.	reclaim 시간이 그대로 애플리케이션이 보는 할당 지연입니다. 그래프 수치는 설명용 예시입니다.
11	**How Does an Allocation Flow Through Reclaim?**	흐름도입니다. 쓸 수 있는 zone이 low 위에 있으면 바로 할당(fast path)합니다. 없으면 kswapd를 깨우고 min 위의 zone을 찾으며, 있으면 거기서 할당하고 kswapd는 계속 reclaim합니다. 모두 min 아래이면 기다릴 수 있는 요청은 direct reclaim 후 재시도하고, atomic 요청은 reserve를 쓰며 실패할 수 있습니다. kswapd는 노드가 균형(zone이 high 도달)이 되면 잠듭니다.	low 위: 즉시 성공, min 위: kswapd가 reclaim하는 동안 성공, 모두 min 아래: direct reclaim.

쪽	슬라이드	내용 요약	기억할 점
12	**Why Does Linux Use Watermarks?**	free page가 0이 될 때까지 기다리면 다음 할당이 막히고 atomic 요청이 실패할 수 있습니다. watermark는 free page reserve를 두어 reclaim을 일찍 시작하게 합니다. 이유는 세 가지입니다: 할당 burst가 reclaim보다 빠를 수 있고, 인터럽트 핸들러 같은 atomic 할당은 sleep할 수 없으며, reclaim 자체(write-back, swap-out)도 free page가 있어야 진행됩니다.	reclaim은 시간이 걸리므로 free가 남아 있을 때 시작해야 합니다.
13	**How Are the Watermark Levels Determined?**	min은 vm.min_free_kbytes에서 오며 zone 크기에 비례해 나눕니다. gap은 vm.watermark_scale_factor에서 오며 low = min + gap, high = min + 2 × gap입니다. vm.lowmem_reserve_ratio는 상위 zone 요청이 낮은 zone으로 fallback할 때 그 zone의 watermark에 보호값을 더합니다. gap이 클수록 kswapd가 일찍 시작해 오래 돌아 cache로 쓰이는 메모리가 줄어듭니다. 단편화가 있으면 watermark boost로 일시적으로 올라갑니다. 값은 /proc/zoneinfo에서 확인합니다.	min은 기다릴 수 없는 요청의 reserve이고, gap은 kswapd가 따라잡을 시간입니다. low와 high는 min과 gap에서 계산됩니다.
14	**What Is the Difference Between MemFree and MemAvailable?**	MemFree는 사용되지 않는 page(zone free page의 합)이고, MemAvailable은 swap 없이 회수할 수 있는 메모리의 추정치입니다. file page(dirty는 write-back 후)와 reclaimable slab을 더하고, anonymous와 Shmem은 swap이 필요해 제외하며, unreclaimable slab도 제외합니다. 예시(16 GB): MemFree 약 1 GB, MemAvailable 약 9 GB. watermark는 둘 중 어느 것도 쓰지 않습니다.	MemFree는 "지금 비어 있는 양", MemAvailable은 "여유 추정"입니다. 수치는 예시이고 커널은 cache와 slab의 일부만 셉니다.
15	**Why Can Total Free Memory Mislead?**	시스템 전체 free가 충분해도 요청이 쓸 수 있는 zone에 free가 없으면 reclaim이 시작됩니다. 예: DMA zone은 low 아래, DMA32와 Normal은 위일 때 DMA 요청은 DMA32나 Normal을 쓸 수 없습니다(fallback은 아래로만). 확인할 것은 요청이 쓸 수 있는 zone의 free이며 /proc/zoneinfo에서 zone별로 봅니다.	전체 free는 "건강해 보이는지"만 보여 줍니다. 값은 설명용 예시입니다.
16	**Checkpoint: Is It Background or Direct Reclaim?**	1부 정리 문제 5개입니다. 상황마다 reclaim 경로(kswapd, direct, 둘 다, 없음)와 이유를 판단합니다. pgscan_direct나 allocstall이 늘면 direct reclaim입니다.	먼저 스스로 답해 본 뒤 마지막 장의 Checkpoint 답과 비교합니다.

2부. 어떤 페이지를 먼저 reclaim하고, 어떻게 해제할까요? (18–33쪽)

이 블록의 핵심은 다음과 같습니다. 비용이 낮고 다시 쓰일 가능성이 낮은 페이지를 LRU list에서 고르고, 한 번의 round에서 목표를 정해 victim을 선택해 해제합니다. anonymous 페이지는 swap이 있어야 내보낼 수 있습니다.

쪽	슬라이드	내용 요약	기억할 점
18	**Which Pages Does Linux Try to Evict First?**	Linux는 reclaim 비용이 낮고 다시 쓸 가능성이 낮은 페이지를 먼저 내보냅니다. 먼저: clean file page, 쓰지 않은 readahead 페이지. 나중: dirty file page(write-back 필요), cold anonymous page(swap-out 필요). 가급적 유지: hot file page, hot anonymous page, 쓰이는 mapped file page. 판단 요소는 재사용 가능성(최근성: 같은 list 안에서 선택), reclaim 비용(list 사이의 스캔 비중), memory pressure(얼마나 세게 스캔할지)입니다.	고정된 순서가 아니라 경향입니다. 기록, 상태, pressure가 모두 영향을 줍니다.

쪽	슬라이드	내용 요약	기억할 점
19	How Are Pages Organized for Reclaim?	reclaim은 전체 메모리가 아니라 LRU(Least Recently Used) list에서 고릅니다. 4개 list(active/inactive × anonymous/file)와 unevictable list 1개, 총 5개입니다. active는 최근에 쓴 page, inactive는 reclaim 후보이며, 주로 inactive의 tail부터 스캔합니다. mlock()된 page는 unevictable에 두고 건너뜁니다. 분리 이유는 비용이 다르기 때문입니다(anonymous는 swap-out 필요, clean file은 그냥 drop).	"4 + 1"과 "inactive tail부터"를 기억합니다.
20	How Does Linux Manage LRU Lists?	각 page descriptor(struct page, 최신 커널은 struct folio)의 lru 필드가 list 연결 고리이므로 page 자체가 list 항목입니다. lruvec 하나가 list 5개의 head와 lru_lock 1개를 가집니다(memory cgroup이 없으면 노드마다 하나, 있으면 cgroup 수 × 노드 수). 원형 이중 연결 리스트이며 새 page는 head에, 가장 오래된 page는 tail에 있습니다. lock 부담은 batch 처리로 줄입니다.	page를 복사하지 않고 연결만 바꿉니다. 클래식 LRU 기준이며 MGLRU는 세부가 다릅니다.
21	What Is a Page's Lifetime on the LRU Lists?	새 page는 inactive에 not referenced로 들어갑니다. 첫 접근에서 스캔이 accessed bit를 지우고 referenced로 표시하고, referenced 상태에서 다시 접근되면 active로 승격합니다. inactive가 작아지면 reclaim이 active tail을 inactive로 deactivate합니다. unmapped file page는 accessed bit를 쓰지 않아 referenced 상태에서 다시 접근되면 바로 active가 됩니다.	두 번 접근해야 active가 됩니다. 클래식 LRU 기준입니다.
22	What Happens in One Reclaim Round?	한 round는 세 단계입니다. 1) 목표 설정: kswapd는 high watermark까지, direct reclaim은 32 pages. 2) victim 선택: 각 list를 scan size(priority로 결정, 12에서 시작)까지 스캔. 3) reclaim: clean victim은 해제하고, dirty는 write-back이나 swap-out 후 해제합니다. 정지 조건을 못 채우면 priority를 낮춰(scan size가 커지게) 다음 round를 반복합니다. 처리는 최대 32 page batch 단위이며, priority는 direct는 0까지, kswapd는 1까지 내려갑니다.	kswapd와 direct reclaim은 같은 round를 돌고 정지 조건만 다릅니다. 32는 일반적인 값입니다.
23	How to Select a Victim for Reclaim?	inactive list의 tail(가장 오래된 쪽)에서 32 page씩 isolate하며, scan size를 다 쓸 때까지 반복합니다. 각 page를 검사해 accessed bit가 0이면 victim(clean은 쓰지 않고 해제, dirty는 쓴 뒤 해제), 1이면 victim이 아니라 이번 round에서 건너뜁니다. victim이 아닌 page는 LRU로 돌아가며 다른 list가 채워 주지 않습니다.	clean/dirty는 victim 여부가 아니라 "어떻게 해제하는가"를 정합니다.
24	What Happens to Each Victim?	clean page는 다른 곳에 사본이 있어 I/O 없이 drop합니다. dirty file page는 reclaim flag를 걸면 flusher가 쓰고, 이후 round에서 write-back이 끝나면 해제합니다. anonymous와 shmem page는 swap slot을 할당해 swap에 쓴 뒤 해제하며, swap이 없으면 이 방법으로 해제할 수 없습니다. slab cache는 같은 round에서 shrinker가 해제하고, write-back 중인 page는 건너뜁니다.	clean은 drop, dirty file은 flusher, anonymous는 swap, slab은 shrinker입니다. 클래식 LRU와 MGLRU가 같습니다.
25	What Are the Working Set and Thrashing?	working set은 프로그램이 계속 다시 쓰는 page이며 근사치이고 시간에 따라 변합니다. 쫓겨난 page가 다시 읽히는 것이 refault이고, 간격이 짧으면 너무 일찍 쫓겨났다는 신호입니다. 메모리가 working set보다 작으면 쫓겨남과 refault가 반복되어 thrashing(높은 I/O, 낮은 성능)이 됩니다. Linux는 최근에 쫓겨난 page가 refault하면 active list로 보냅니다.	refault는 reclaim이 너무 일찍 내보냈다는 신호이고, 반복되면 thrashing입니다.

쪽	슬라이드	내용 요약	기억할 점
26	**Why Does Linux Need MGLRU?**	MGLRU(Multi-Generational LRU)는 클래식 active/inactive를 대체하는 reclaim 구현입니다. 두 list는 page 나이를 거칠게만 보여 주므로, 각 page를 마지막 접근 시점에 맞는 generation(2~4개)에 둡니다. 접근한 page는 가장 새 generation으로 가고, 시간이 지나면 generation이 나이 들고, reclaim은 가장 오래된 generation부터 합니다. refault와 스캔 비용이 줄 수 있고(워크로드에 따라 다름), Linux 6.1부터 mainline이며 기본 사용 여부는 설정과 배포판에 따릅니다. /sys/kernel/mm/lru_gen/enabled로 확인합니다.	두 list 대신 여러 generation으로 나이를 더 세밀하게 구분합니다.
27	**Why Does Linux Need Swap?**	live anonymous page는 backing file이 없어 swap이 있어야 내보낼 수 있습니다. swap이 없으면 cold anonymous page가 RAM에 남아 cache가 줄고 OOM이 빨라집니다. swap이 있으면 cold page를 swap area(disk 또는 zram)로 옮겨 RAM을 hot page와 cache에 씁니다. 대가: swap-in은 major fault이며 마이크로초~밀리초(RAM은 나노초), 과도한 swap은 thrashing으로 시스템을 느리게 합니다.	swap은 RAM의 대체가 아닙니다. hot working set은 RAM에, cold한 나머지는 swap에 둡니다.
28	**Which Swap Backend Fits: Disk, zram, or zswap?**	세 가지 모두 같은 swap-out 경로로 page를 받습니다. disk swap은 용량이 크고 영속적이지만 swap I/O마다 저장장치를 기다립니다. zram은 RAM 안의 압축 swap으로 disk I/O가 없고 CPU를 쓰며 용량이 RAM에 묶입니다. zswap은 disk swap 앞의 압축 cache로, hit이면 빠르고 쫓겨날 때만 disk로 갑니다. swapon --show로 장치, 크기, 우선순위(PRIO)를 봅니다. 높은 PRIO가 먼저 쓰입니다.	용량(disk), disk I/O 회피(zram), 압축 cache(zswap) 중 trade-off로 고릅니다.
29	**How Does zram Work as Swap?**	zram은 RAM에 있는 block device를 swap으로 씁니다. swap-out은 page를 압축해 zram에 저장하고, page fault가 나면 압축을 풀어 복원합니다. 압축이 잘 되는 데이터는 메모리를 많이 아끼고, 잘 안 되는 데이터는 거의 아끼지 못합니다(예: 100 MB가 45 MB로, 설명용). 설정은 disksize, mem_limit, comp_algorithm(lz4, zstd)이고 zramctl로 DATA, COMPR, TOTAL을 확인합니다.	압축에는 CPU가 듭니다. 효과는 데이터의 압축률, CPU와 RAM 여유에 따라 달라집니다.
30	**How Does swappiness Bias Reclaim?**	vm.swappiness는 reclaim이 anonymous 대신 file page를 얼마나 선호하는지의 편향입니다(swap할 메모리 비율이 아닙니다). 범위는 0~200(5.8 이상, 기본 60)이며 anonymous 가중치는 swappiness, file 가중치는 200 − swappiness입니다. 낮은 값(예: 10)은 느린 swap(disk, SD card)이나 flash 마모가 걱정될 때, 높은 값(100~200)은 zram 같은 빠른 swap일 때 맞습니다. 0이라도 classic LRU에서는 메모리가 매우 부족하면 swap이 시작될 수 있습니다(커널 버전에 따라 다름).	swappiness는 편향일 뿐입니다. 결과는 LRU 상태, refault, swap 여유에도 달라집니다. 예시값은 워크로드별로 조정합니다.
31	**What Is a Shrinker and How Does It Work?**	shrinker는 커널 cache(dentry, inode 등)가 등록하는 callback이며 thread가 아닙니다. kswapd와 direct reclaim이 LRU list를 스캔한 뒤 호출합니다. count_objects()는 해제 가능한 객체 수를, scan_objects(n)는 최대 n개를 해제합니다. 해제된 slab page는 LRU page와 같은 목표에 포함되고, scan size는 priority를 따르되 LRU와 별도로 해제 가능한 객체 수에 따라 정해집니다. priority가 내려가면 둘 다 scan size가 커집니다.	LRU 다음에 shrinker가 같은 round에서 같은 목표를 향해 일합니다.

쪽	슬라이드	내용 요약	기억할 점
32	When Does Reclaim Make No Progress?	많이 스캔하지만 해제는 적을 때입니다. 원인은 세 가지입니다: 회수 가능한 pool이 작음(대부분 locked, pinned, 사용 중), reclaim 비용이 큼(dirty는 write-back, anonymous는 swap 공간 필요), hot page가 바로 다시 쓰임(헛수고). 단계: 압박 지속 → priority 번호가 내려가 scan size 증가 → direct reclaim, write-back, swap → 그래도 적으면 allocation stall이나 재시도 → OOM kill 가능. 정확한 재시도는 GFP flag와 상태에 따라 다릅니다.	실패가 이어지면 allocation stall, OOM killer로 갑니다 (10주차).
33	Checkpoint: Which Page Goes First?	2부 정리 문제 6개입니다. 페이지 종류(clean file, dirty file, anonymous, refault, mlock/pinned)별로 reclaim 여부와 이유를 판단합니다.	먼저 스스로 답해 본 뒤 마지막 장의 답과 비교합니다. 순서는 경향이지 규칙이 아닙니다.

3부. 메모리 압박을 어떻게 관찰하고 제어할까요? (35–48쪽)

이 블록의 핵심은 다음과 같습니다. PSI가 심각도를, vmstat 카운터가 reclaim 경로를, memory cgroup이 소유자와 한도를 보여 줍니다. 튜닝 전에 진단하고, 설정은 비용을 옮길 뿐이므로 한 번에 하나씩 바꿔 측정합니다.

쪽	슬라이드	내용 요약	기억할 점
35	What Is PSI?	PSI(Pressure Stall Information)는 자원이 부족해 task가 얼마나 stall했는지를 보고하는 커널 기능입니다. 사용량은 task가 느끼는 지연을 보여 줄 수 없지만 stall 시간은 보여 줍니다. 대상은 CPU, memory, I/O이고 idle 시간은 세지 않습니다. some은 적어도 한 task가 stall된 시간의 비율, full은 모든 non-idle task가 동시에 stall된 시간의 비율입니다(예: 10칸 중 6칸이면 some 60%, 1칸이면 full 10%). memory stall은 reclaim, compaction, refault, swap-in을 기다리는 시간입니다.	some은 "누군가 기다림", full은 "모두 기다려 진행 없음"입니다.
36	How Do We Read the PSI Files?	/proc/pressure/cpu, memory, io가 같은 형식입니다. some 줄과 full 줄이 있고 각 줄은 avg10, avg60, avg300(최근 10, 60, 300초의 stall %)과 total(부팅 이후 누적 stall µs)을 가집니다. cpu의 full 줄은 항상 0입니다. 빠르게 보려면 avg(avg10이 avg300보다 크면 상승 중), 정확한 비율은 (끝 total − 시작 total) ÷ 경과 시간입니다. 예시 수치는 설명용입니다.	어느 자원이 stall하는지 세 파일로 찾고, avg는 빠른 확인, total 차이는 정확한 비율입니다.
37	How Do Programs Monitor Memory Pressure With PSI?	/proc/pressure/ 파일은 로그가 아니라 읽을 때마다 커널이 현재 값을 만드는 가상 파일입니다. 프로그램은 두 방법을 씁니다. polling은 주기적으로 읽어 avg10이나 total 차이로 계산해 한도와 비교합니다. trigger는 /proc/pressure/memory를 O_RDWR | O_NONBLOCK으로 열고 조건(예: some 150000 2000000 = 2초 window에서 150 ms stall)을 write한 뒤 poll()로 POLLPRI 이벤트를 기다립니다. window는 500 ms~10 s입니다.	trigger는 조건을 한 번 쓰고 커널의 알림을 기다립니다. 조건은 some/full, stall 시간, window입니다.
38	How Do We Diagnose Memory Pressure?	튜닝 전에 진단합니다. 세 가지를 순서대로 묻습니다. 1) 얼마나 나쁜가: /proc/pressure/memory(some, full, avg, total). 2) 어떤 경로인가: /proc/vmstat의 allocstall_*, pgscan_*/pgsteal_*, pswpout/pswpin. 3) thrashing인가: workingset_refault_file/anon(5.9 이상, 이전은 workingset_refault). vmstat 카운터는 누적이므로 두 sample을 비교합니다. 이후 수요 줄이기, headroom 확보, 정책 변경 중에서 고르고 PSI로 다시 확인합니다.	심각도는 PSI, 경로는 vmstat, thrashing은 refault로 봅니다.

쪽	슬라이드	내용 요약	기억할 점
39	**Observation: How Bad?**	같은 file reader를 네 상황에서 실행한 실측(2 CPU, 8 GB RAM, swap 없음 테스트 VM)입니다. 시작과 끝의 total을 빼 실행 시간으로 나누면 비율입니다(예: (9,576,671 − 149) ÷ 25 s = 38.3%). 1) 정상: some 0.0%. 2) CPU 경합(6개 busy-loop): 읽기가 느려도 memory some 0.0%, cpu some 99.4%. 3) memory pressure(다른 task가 6 GB 점유): memory some 38.3%, full 37.9%. 4) pressure + busy-loop 1개: some 32.5%, full 26.2%, cpu some 24.7%.	느리다고 모두 메모리 문제는 아닙니다. 다른 task가 계속 실행되면 some이 full보다 큽니다.
40	**Observation: Which Path?**	두 상황을 vmstat와 memory PSI로 비교한 실측입니다(값은 두 sample의 차이 ÷ 시간). kswapd only(400 MB/s file reader): memory some 0.3%, pgscan_kswapd 91,376/s, pgscan_direct 38/s, allocstall 0.1/s로 kswapd가 따라잡습니다. direct reclaim(8.8 GB 할당, swap on): some 76.7%, pgscan_direct 405,117/s, allocstall 2,484/s로 task가 직접 reclaim하며 대부분의 시간을 기다립니다. some이 높은데 pgscan_direct와 allocstall이 낮으면 refault, swap-in 등 다른 대기입니다.	pgscan_direct와 allocstall은 direct reclaim 여부, memory some은 대기 시간을 보여 줍니다. kswapd의 reclaim 시간도 some에 조금 포함됩니다.
41	**Observation: Thrashing or Swap Wait?**	두 상황을 더 봅니다(실측, 같은 VM). thrashing(6 GB 점유, 2 GB file 재읽기): workingset_refault_file 834,653/s, memory some 40.5%, pswpout과 pswpin은 0입니다. swap-in(swap된 page 접근): pswpin 106,520/s, pswpout 100,592/s, some 11.0%로 swap 읽기를 기다립니다. pswpout만 높으면 pressure의 결과이지 그 자체로 대기는 아닙니다. some이 높은데 refault, pswpin, direct reclaim이 낮으면 compact_stall 등 다른 원인을 봅니다.	refault는 thrashing, pswpin은 swap 읽기 대기를 보여 주고, pswpout만으로는 압박이 있다는 사실만 알 수 있습니다.
42	**What Is a Memory Cgroup?**	cgroup(control group)은 프로세스를 묶어 자원을 하나의 단위로 계산하고 제한하는 커널 기능이고, memory cgroup은 이를 메모리에 적용합니다. 그룹은 tree를 이루며 부모의 한도가 자식에도 적용됩니다. 그룹마다 사용 카운터(memory.current, memory.stat), 제어(memory.min/low/high/max), 자체 LRU list(lruvec), 상태 파일(memory.events, memory.pressure)이 있습니다. cgroup v2에서 /sys/fs/cgroup 아래 파일로 다루며, 예: mkdir, cgroup.procs에 PID 쓰기, memory.high에 값 쓰기입니다.	그룹별로 reclaim과 pressure를 다룰 수 있습니다. 부모 cgroup에서 memory controller가 켜져 있어야 합니다.
43	**How Do Memory Cgroup Controls Steer Reclaim?**	그룹의 사용량(memory.current)에 따른 네 가지 제어입니다 (zone watermark와 무관). memory.min은 hard 보호 (reclaim이 건드리지 않음, 더 reclaim할 것이 없으면 OOM), memory.low는 best-effort 보호(다른 그룹에 low 초과분이 있으면 건너뜀), memory.high는 초과하면 그 그룹을 reclaim하고 task를 throttle(OOM 없음), memory.max는 hard 한도(reclaim이 실패하면 그룹 안의 task가 OOM kill)입니다. 일반 순서는 min ≤ low ≤ high ≤ max입니다. throttle은 할당한 task가 그룹의 page를 직접 reclaim하고 잠시 sleep할 수 있다는 뜻입니다.	min과 low는 보호, high와 max는 제한입니다. 효과는 격리(초과 그룹을 먼저 reclaim)와 예측 가능성입니다.
44	**What Is the Difference Between memory.high and memory.max?**	memory.high는 soft 한도입니다. 초과하면 그룹의 task가 throttle되고 그룹이 reclaim되며, high 자체로는 OOM kill이 없습니다. memory.max는 hard 한도입니다. 할당이 max에 닿으면 그룹 안에서 direct reclaim을 하고, 그래도 자리가 없으면 그룹 안의 task 중에서만 OOM kill 후보가 됩니다. 함께 쓰며 high를 max보다 낮게 둡니다(예: high 400M, max 500M).	high를 낮게 두면 OOM kill보다 throttle이 먼저 일어납니다.

쪽	슬라이드	내용 요약	기억할 점
45	**Observation: How Is a Memory Cgroup Controlled?**	한 프로세스가 700 MB를 할당할 때 high 400M, max 500M(swap 없음)인 그룹의 변화입니다(설명용 값). 400M 아래: memory.events high 0, max 0, pressure some ≈ 0. high 초과(≈ 450M): high 1,240, some ≈ 18. max(500M): max 18, oom_kill 1, some ≈ 25. events는 횟수(high = throttle 횟수, max = 한도 도달 횟수, oom_kill = 죽은 task 수)이고 memory.pressure는 /proc/pressure/memory와 같은 형식입니다. 시스템 MemAvailable은 약 6 GB로 충분합니다.	시스템 메모리가 충분해도 그룹 파일에서 그 그룹의 한도 이벤트와 stall을 볼 수 있습니다.
46	**What Can We Tune, and What Does Each Setting Cost?**	설정은 비용을 이동시킵니다. min_free_kbytes(min 상향): burst를 위한 더 큰 reserve ↔ 사용 가능한 메모리 감소와 이른 reclaim. watermark_scale_factor(gap 확대): kswapd가 일찍 시작해 direct reclaim이 줄 수 있음 ↔ file cache 감소. swappiness: file이나 anonymous 보호 ↔ swap I/O 증가 또는 cache 손실. zram: disk I/O 없는 swap ↔ 압축에 CPU와 RAM. memory.high/low: 그룹 간 격리 ↔ 그룹 throttling.	한 번에 하나씩 바꾸고 PSI와 워크로드 지연 시간으로 전후를 비교합니다. 기본값은 출발점이고 효과는 커널 버전과 환경에 따라 다릅니다.
47	**What Should webOS-Class Devices Do About It?**	작은 DRAM(예: 2~4 GB)을 앱, 프레임워크, compositor/UI, page cache, 커널이 함께 써서 메모리가 빨리 부족해지고, direct reclaim을 기다리는 task는 앱 실행 지연이나 UI jank로 보입니다. 네 가지 설계 방향입니다. headroom(launch burst에 맞춘 watermark), 백그라운드 reclaim(kswapd가 stall 전에 CPU 확보), component 제한(서비스별 cgroup 한도와 보호), 저렴한 swap 계층(CPU와 메모리 예산이 허용하면 zram). 각 방향에는 비용이 있습니다.	특정 webOS 구현의 설명이 아니라 일반 설계 고려사항입니다. 대상 기기에서 PSI와 launch latency로 검증합니다.
48	**Checkpoint: What Do These Signals Mean?**	3부 정리 문제 7개입니다. 신호 조합(PSI, vmstat, memory.events)이 의미하는 것과 다음에 확인할 것을 판단합니다. PSI는 심각도, 카운터는 경로, cgroup 이벤트는 소유자를 보여 줍니다.	먼저 스스로 답해 본 뒤 마지막 장의 답과 비교합니다. 한 값만으로는 원인을 확정하기 어렵습니다.

Week 9 한 장 요약과 핵심 용어 (49쪽)

한 줄 요약. memory pressure는 할당 비용의 상승이고, watermark가 kswapd(low 아래)와 direct reclaim(min 아래)을 시작시키며, Linux는 비용이 낮고 재사용이 적은 페이지부터 회수합니다. PSI, 카운터, cgroup 파일로 압박을 측정한 뒤 튜닝하고, 회수 가능한 메모리가 줄면 OOM(10주차)에 가까워집니다.

쪽	슬라이드	내용 요약	기억할 점
49	**Summary of Week 9**	세 칸으로 정리합니다. 1) Fundamentals: pressure는 할당 비용의 상승, min/low/high watermark, kswapd와 direct reclaim, zone별 pressure. 2) Policy: 비용과 재사용이 먼저 회수할 page를 정함, active/inactive LRU와 MGLRU, working set과 refault, file vs. anonymous와 swap, zram. 3) Observe and Impact: PSI, cgroup memory.high/max와 memory.events, thrashing과 튜닝 trade-off, 작은 메모리 기기의 headroom과 한도. 다음 시간(Week 10)은 reclaim이 따라가지 못할 때의 OOM입니다.	발생(watermark) → 선택(LRU, swap) → 관찰(PSI, 카운터, cgroup) → 튜닝의 흐름으로 기억합니다. 회수 가능한 메모리가 줄수록 OOM이 가까워집니다.

용어	의미
memory pressure	새 할당이 더 많은 reclaim 일이나 대기를 요구하는 정도(할당 비용의 상승)
reclaim	사용 중인 메모리를 해제해 free memory를 확보하는 작업
slow path, fast path	할당 경로. fast path는 low 위 free page로 즉시 할당, slow path는 reclaim 후 할당

용어	의미
watermark (min, low, high)	zone의 free page 기준선. min은 reserve, low는 kswapd 시작, high는 kswapd 정지
vm.min_free_kbytes	min watermark를 정하는 sysctl
vm.watermark_scale_factor	watermark 사이 gap의 크기를 정하는 sysctl(low = min + gap, high = min + 2 × gap)
kswapd	노드마다 하나인 백그라운드 reclaim 커널 스레드
direct reclaim	할당 task가 자기 context에서 하는 reclaim. task가 기다림
atomic allocation (GFP_ATOMIC)	sleep할 수 없는 문맥의 할당. min 아래 reserve를 쓰거나 실패
MemFree, MemAvailable	사용되지 않는 page의 양과, swap 없이 회수 가능한 메모리의 추정치
LRU list	Least Recently Used list. active/inactive × anonymous/file 4개와 unevictable 1개
lruvec	LRU list 5개의 head와 lru_lock을 가진 구조(노드나 cgroup마다)
accessed bit	PTE에서 CPU가 접근 시 켜는 비트. 스캔이 확인하고 지움
victim	한 reclaim round에서 해제 대상으로 선택된 page
priority, scan size	스캔할 양을 정하는 값(12에서 시작, 낮아지면 scan size가 커짐)
write-back, flusher	dirty file page를 파일에 쓰는 일과 이를 수행하는 커널 스레드
working set, refault	프로그램이 계속 쓰는 page 집합과, 쫓겨난 page가 다시 읽히는 일
thrashing	refault가 반복되어 I/O가 치솟고 성능이 떨어지는 상태
MGLRU	Multi-Generational LRU. generation으로 page 나이를 구분하는 reclaim 구현(Linux 6.1+)
swap, swap-in/out	anonymous page를 저장장치나 압축 RAM에 내보내고(out) 다시 읽는(in) 것
zram, zswap	RAM 안의 압축 swap 장치, disk swap 앞의 압축 cache
vm.swappiness	anonymous와 file reclaim 사이의 편향(0~200, 기본 60)
shrinker	커널 cache가 등록해 reclaim이 호출하는 해제 callback(count_objects, scan_objects)
PSI, some, full	Pressure Stall Information. 일부 task가 stall된 시간과 모든 non-idle task가 stall된 시간의 비율
avg10/60/300, total	최근 10/60/300초 평균 stall %와 부팅 이후 누적 stall µs
allocstall, pgscan_direct	direct reclaim이 일어난 할당 횟수, direct reclaim이 스캔한 page 수(vmstat)
pswpin, pswpout	swap에서 읽은 page 수, swap에 쓴 page 수(vmstat)
workingset_refault_file/anon	쫓겨난 뒤 다시 읽힌 file/anonymous page 수(vmstat)
memory cgroup	프로세스 그룹의 메모리를 계산하고 제한하는 cgroup v2 controller
memory.min, low	그룹 보호(hard, best-effort)
memory.high, max	그룹 한도(soft: throttle과 reclaim, hard: 초과 불가, 실패 시 OOM kill)
memory.events, memory.pressure	그룹의 한도 이벤트 횟수와 그룹별 PSI(같은 형식)
OOM killer	reclaim으로 해결하지 못할 때 task를 종료하는 최후 수단(Week 10)

Checkpoint 문제와 답 (16, 33, 48쪽)

먼저 슬라이드의 질문을 보고 스스로 답해 본 뒤, 아래 답과 비교해 보시기 바랍니다. 정답이 하나만 있는 문제는 아니며, 판단의 이유(trigger와 문맥, 비용과 재사용, 신호의 조합)를 설명할 수 있으면 됩니다.

16쪽. Is It Background or Direct Reclaim?

상황	어떤 reclaim 경로?	이유
쓸 수 있는 zone 중 low 위는 없지만 min 위는 있다.	**백그라운드 reclaim (kswapd)**	slow path가 kswapd를 깨웁니다. min 위이므로 할당은 성공할 수 있습니다.
대기할 수 있는 task가 할당하는데 쓸 수 있는 모든 zone이 min 아래다.	**direct reclaim (요청한 task)**	task가 자기 context에서 reclaim하며 멈출 수 있습니다.
kswapd가 돌고 있는데 free page가 계속 줄고 allocstall_normal이 늘어난다.	**둘 다: kswapd와 direct reclaim**	kswapd가 따라가지 못해 task가 직접 reclaim합니다.
MemFree는 낮지만 MemAvailable은 높다.	**즉시 압박 없음**	회수 가능한 메모리가 많습니다. MemFree가 낮은 것만으로는 압박이 아닙니다.
쓸 수 있는 모든 zone이 min 아래일 때 인터럽트 핸들러가 GFP_ATOMIC으로 할당한다.	**direct reclaim 없음 (sleep 불가)**	atomic 문맥은 sleep할 수 없어 reserve를 쓰거나 실패합니다. kswapd는 깨어납니다.

마무리 질문: trigger(low 아래, min 아래)와 문맥(대기 가능 여부)으로 reclaim 경로를 구분하고, MemFree가 낮은 것만으로는 압박이 아님을 설명할 수 있나요?

33쪽. Which Page Goes First?

페이지 / 객체	reclaim되는가?	이유
오래 쓰지 않은 clean file page (예: 오래된 readahead 데이터)	**예, 가장 먼저**	I/O 없이 바로 drop합니다.
dirty file page (수정됐지만 아직 write-back 전)	**나중에**	해제하려면 먼저 write-back해야 합니다.
cold anonymous page, swap 있음	**예, 비용을 치르고**	swap-out(disk I/O나 압축)이 필요합니다.
cold anonymous page, swap 없음	**아니요**	swap이 없으면 anonymous page는 reclaim할 수 없습니다.
방금 refault한 page (working set)	**예, 가장 나중에**	다시 필요해졌으므로 active에 유지하며, 압박이 계속될 때만 해제합니다.
mlock되었거나 pinned된 page, 사용 중인 커널 메모리	**아니요**	unevictable이거나 사용 중이라 reclaim이 해제할 수 없습니다.

마무리 질문: reclaim이 "해제 비용이 낮고 다시 쓰일 가능성이 낮은" page를 먼저 고르는 이유와, 그 순서가 고정 규칙이 아닌 이유를 설명할 수 있나요?

48쪽. What Do These Signals Mean?

신호 (예시)	읽는 법	다음에 확인할 것
PSI some 낮음, allocstall 변화 없음	**의미 있는 memory stall 없음**	CPU와 I/O를 확인
pgscan_direct 증가, pgsteal 변화 없음	**스캔만 하고 진전 없음**	pinned나 dirty page, swap 공간 없음

신호 (예시)	읽는 법	다음에 확인할 것
pgscan_kswapd 증가, pgscan_direct 변화 없음, PSI some 낮음	**kswapd가 따라잡고 있음**	추세를 지켜봄
workingset_refault_file 증가, pswpin 변화 없음	**file cache thrashing**	file working set 대 RAM
pswpin 증가, workingset_refault_anon 증가	**swap-in 대기**	swap 장치와 anonymous 메모리
memory.events: max 증가	**그룹이 memory.max에 도달**	한도를 올리거나 사용량을 줄임
memory.events: oom_kill > 0	**cgroup 안에서 OOM kill 발생**	OOM 처리 (Week 10)

마무리 질문: PSI(심각도), vmstat(경로), cgroup 이벤트(소유자)를 함께 읽어야 하는 이유를 설명할 수 있나요?